Intain Campus FinTech Challenge 2026

Full Stack Track Problem Statement

Loan Data Verification Copilot

Build an AI-assisted full-stack console that turns messy loan records into validated, traceable, trusted data.

1. Challenge Summary

Financial platforms depend on loan-level data. But loan data rarely arrives clean. It may come from CSV exports, APIs, servicing systems, origination systems, or manually maintained spreadsheets.

The challenge is to build a Loan Data Verification Copilot: a full-stack application that ingests messy loan records, detects data-quality issues, uses AI assistance to explain and resolve exceptions, and creates a traceable verified record.

This is not a structured-finance challenge. Participants do not need to understand securitization, asset-backed securities, waterfall models, borrowing bases, or capital markets.

2. Overview

Can you build a system that takes a messy loan tape and turns it into a clean, verified, auditable dataset?

- Data ingestion and normalization
- Backend validation and exception handling
- Full-stack workflows and role-based interfaces
- Audit logs, record hashing, and traceability
- AI-assisted review and agentic coding discipline
- API design, deployment, and demo readiness

3. Business-Adjacent Context

A loan tape is simply a table of loan records. Each row represents one loan; each column describes attributes such as loan amount, interest rate, origination date, maturity date, borrower state, payment status, document availability, and last updated date.

Platforms that work with loans need to trust this data before using it for analytics, reporting, review, or operational workflows. The challenge is to build the verification layer that makes this possible.

4. Public Data Source URLs

The organizer should provide a curated synthetic dataset for default judging. Participants may optionally use the public sources below for stretch features, public-data connectors, or alternative ingestion pipelines, subject to the relevant source terms.

Source	Actual URL	What participants can get	Access / usage note
Fannie Mae Single-Family Loan Performance Data	https://capitalmarkets.fanniemae.com/credit-risk-transfer/single-family-credit-risk-transfer/fannie-mae-single-family-loan-performance-data	Acquisition data, performance data, sample file, glossary / file layout, Data Dynamics access, and APIs.	Registration and terms acceptance required. Best used as schema inspiration and optional data source.

Fannie Mae Data Dynamics access page	https://datadynamics.fanniemae.com/data-dynamics/#/reportMenu;category=Loan_Performance	Direct portal for Single-Family Loan Performance Data downloads.	Registration required. Use only if participants want to work with actual Fannie Mae data.
Fannie Mae sample file / file layout	Start from the main Fannie Mae page above and use links titled "Sample File" and "CRT Glossary and File Layout."	Sample acquisition/performance file and schema definitions.	Useful for designing the simplified synthetic schema.
Freddie Mac Single-Family Loan-Level Dataset	https://www.freddiemac.com/research/datasets/sf-loanlevel-dataset	Loan-level mortgage performance data, standard dataset by year, non-standard dataset, sample files, user guide, file layout, and release sample files.	Registration/sign-in through Clarity Data Intelligence required. Free for non-commercial, academic/research, and limited use subject to terms.
Freddie Mac Clarity Data Intelligence download page	https://claritydownload.fmapps.freddiemac.com/	Direct access portal for Freddie Mac Single-Family Loan-Level Dataset downloads.	Registration/sign-in required. Use for teams that want to download Freddie Mac data directly.
Freddie Mac file layout and sample files	Start from the Freddie Mac dataset page and use the links under "Upcoming Disclosure Changes" and "Additional Resources."	General user guide, file layout, file headers, sample files, FAQs, release notes.	Useful for schema reference and sample-file design.

Recommendation: use a synthetic organizer-provided dataset for judging. Use public datasets as references and optional stretch sources because official datasets may be large, registration-gated, and subject to source-specific terms.

5. Organizer-Provided Data Package

The default competition package should be synthetic, but modeled on public loan-level schemas. It should not require participants to register with public data portals or interpret complex data dictionaries during the hackathon.

- loan_tape.csv - primary dataset with 1,000 to 5,000 loan records.
- servicer_update.csv - second-source update file with partial or conflicting loan information.
- document_manifest.csv - mock document availability by loan ID.
- validation_rules.json - configurable rules for validating records.
- users.json - mock users and roles.
- expected_exception_sample.csv - small sample of known exceptions for orientation.

6. Example Dataset Fields

- loan_id
- borrower_id
- loan_type
- origination_date
- maturity_date
- original_principal
- current_balance
- interest_rate
- term_months
- borrower_state
- loan_purpose
- credit_grade
- employment_length
- income_band
- payment_status

- days_past_due
- servicer_name
- last_payment_date
- last_updated_at
- document_status
- source_system

7. Intentional Data Issues

- Missing loan IDs
- Duplicate loan IDs
- Duplicate borrower + loan amount + origination date combinations
- Invalid date formats
- Maturity date before origination date
- Negative principal balance
- Current balance greater than original principal
- Interest rate outside expected range
- Payment status inconsistent with days past due
- Missing document status
- Conflicting values between loan_tape.csv and servicer_update.csv
- Stale records based on last_updated_at
- Invalid state codes
- Suspiciously repeated borrower records
- Loans marked closed but still showing positive balance

8. What Participants Must Build

Module A: Data Ingestion

- Upload CSV file
- Parse records
- Store raw uploaded data
- Normalize records into an internal schema
- Show upload summary
- Identify failed import rows
- Preserve source-file lineage

Module B: Validation Engine

- Required fields present
- Valid dates and numeric values
- No negative principal or balance
- Maturity date after origination date
- Current balance not greater than original principal
- Valid payment status
- Duplicate loan detection
- Required document status available
- Stale record detection

Module C: Exception Queue

- View exceptions
- Filter by exception type and severity

- Search by loan ID or borrower ID
- Open loan detail view
- Add review comments
- Approve, reject, or request correction
- Edit allowed fields
- Track reviewer action history

Module D: AI Review Assistant

- Explain why a record failed validation
- Suggest likely corrections
- Compare conflicting records and recommend reliable values
- Generate reviewer notes
- Classify exception severity
- Summarize a batch of exceptions
- Generate validation rules or tests from natural language

Module E: Verified Loan Record

- Canonical loan data
- Source file reference
- Validation result
- Reviewer decision, if any
- AI recommendation, if used
- Verification timestamp
- Verified-by user
- Record hash

Module F: Audit Trail

- File uploaded
- Loan record imported
- Validation executed
- Exception created
- AI recommendation generated
- Reviewer comment added
- Field edited
- Loan approved or rejected
- Verified record created
- Verified record exported

Module G: Dashboards

- Data Operator dashboard: upload, import history, validation summary, corrections needed
- Reviewer dashboard: exception queue, AI panel, pending decisions, recent decisions
- Data Consumer dashboard: verified records, data-quality score, verification history, export and audit trail

Module H: Verified Records API

- GET /loans
- GET /loans/:id
- GET /exceptions
- GET /verified-loans
- GET /verified-loans/:id
- GET /audit/:loanId
- GET /summary

9. Required AI Controls

- Show AI recommendation separately from final human decision.
- Allow reviewer to accept, reject, or edit AI suggestions.
- Log AI-generated suggestions in the audit trail.
- Show prompt, model, timestamp, or equivalent metadata where feasible.
- AI output must not silently change data.

10. Agentic Coding Requirement

Participants must demonstrate how they used AI or agentic coding tools during development. This includes AI used to build, test, refactor, document, or debug the system - not only AI features inside the application.

- Tools used: Cursor, Claude Code, GitHub Copilot, ChatGPT, Gemini, OpenAI API, LangChain, local LLMs, etc.
- Use cases: architecture, API design, schema design, validation-rule generation, UI generation, test generation, debugging, code review, refactoring, documentation, demo script.
- Prompt examples: include 5 to 10 representative prompts.
- Human review process: explain how AI-generated code was reviewed, tested, and corrected.
- AI-generated code percentage estimate: rough estimate is acceptable.
- What was rejected: at least two examples where AI output was wrong, unsafe, inefficient, or unsuitable.
- Lessons learned: explain where AI helped most and where human engineering judgment was necessary.

11. Technical Requirements

- Frontend application
- Backend API
- Database persistence
- File ingestion
- Validation logic
- Exception workflow
- AI-assisted feature
- Audit trail
- Verified-record creation
- Deployment or local runnable setup

Suggested Stacks

- React / Next.js / Vue / Angular frontend
- Node.js / Express / NestJS backend
- Python / FastAPI backend
- PostgreSQL / MySQL / MongoDB / SQLite database
- Prisma / SQLAlchemy / TypeORM / Mongoose optional
- OpenAI / Anthropic / Gemini / local LLM optional
- LangChain / LlamaIndex / CrewAI / AutoGen optional
- Cursor / Claude Code / GitHub Copilot optional

12. Expected Deliverables

- GitHub repository with complete source code.
- Working application: hosted deployment or local runnable version.
- README with setup instructions, environment variables, and run commands.
- Demo video: maximum 5 minutes.

- Architecture note: 1 to 2 pages covering system design, data model, API design, validation engine, AI feature, audit trail, and trade-offs.
- AI Development Log: required.
- Test credentials for Data Operator, Reviewer, and Data Consumer.
- Sample output: verified loan dataset and audit trail export.

13. Full Stack Roles

Backend / Data / AI Integration

- API design
- Database schema
- CSV ingestion
- Validation engine
- AI review endpoint
- Audit trail
- Hashing
- Verified records API
- Backend deployment

Frontend / Workflow / UX

- Upload UI
- Role-based views
- Dashboards
- Exception queue
- AI assistant panel
- Loan detail screen
- Audit trail viewer
- Demo polish

14. Judging Criteria

Category	Points	What judges should look for
Full-Stack Product Completeness	20	Working frontend/backend, CSV ingestion, persistence, runnable application, end-to-end demo.
Backend Architecture and Data Modeling	15	Clean schema, good APIs, modular validation, clear lifecycle, error handling.
Frontend Workflow and UX	15	Intuitive upload, usable exception queue, dashboards, role-based views.
AI Feature Quality	15	Relevant AI workflow, visible recommendations, human control, logged outputs.
Agentic Coding Demonstration	15	AI development log, prompt evidence, human review, examples of rejected AI output.
Traceability and Auditability	10	Raw-to-verified lineage, audit trail, hashing, understandable record history.
Demo Quality	10	Clear five-minute walkthrough, working software, architecture choices, honest limitations.

15. Five-Minute Demo Flow

- Log in as Data Operator.
- Upload a messy loan tape.
- See import and validation summary.
- Open records with validation failures.
- Log in as Reviewer.
- Use AI to explain an exception.
- Accept, edit, or reject AI recommendation.
- Approve or reject loan records.
- Create verified loan records.
- Log in as Data Consumer.
- View verified records dashboard.
- Open one loan and inspect audit trail.
- Show API response for verified records.
- Show AI Development Log.

16. Out of Scope

- Real structured-finance analytics
- Securitization logic
- Borrowing-base calculations
- Real OCR
- Real blockchain deployment
- Real underwriting decisions
- Credit scoring models
- Payment workflows
- Production-grade security
- Regulatory compliance engine